DCA Investing Smart Contract (Interval-Based Buying) with Automation

TA: Ignat Melnikov
Team: Lyubov Lazareva, Roman Farforov, Ruslan Ziyazetdinov

Intro to the project

Dollar-Cost Averaging is an investment strategy where you invest a fixed amount of money at regular intervals, regardless of the asset price.

Problem and Idea

- Manual recurring purchases are inconvenient
- Users need automation and basic safety checks

Our goal was to build a simple end-to-end prototype that shows how the strategy can be automated with blockchain infrastructure.

/start 23:49

DCA MVP bot commands:
/start
/help
/plans
/plan <id>
/pause <id>
/resume <id>
/cancel <id>
/topup <id> <amount>
/withdraw <id> <amount>
/create
/update <id>

Configured wallet:
0x59Fc5C3F24eAaa266c2B0642461D3c58842b46B2
Create flow asks for: amountPerInterval, totalBudget, intervalSeconds, slippageBps, recipient, startTime.
Update flow asks for: amountPerInterval, intervalSeconds, slippageBps. 23:49

Enter the 'Amount Interval' in the USDC. 08:33

0.001 08:33

Enter 'totalBudget' in the USDC 08:34

0.01 08:34

Enter 'intervalSeconds' as an integer. 08:34

3600 08:34

Enter 'slippageBps as an integer. 08:34

300 08:34

Enter the recipient's address or 'me'. 08:35

me 08:35

Enter the start time: 'now', unix timestamp, or ISO datetime. 08:35

now 08:35

Plan #0 created.
Tx:
0x81e2c4d1f66021a5bbbb8d93fdf03cc094cca2cfd0b6ad11ff0c217bf9d91045 23:57

Architecture Overview

- **Smart Contract:** DCAPlanManager.sol handles logic, budgets, and swaps.
- **Executor Service:** Node.js service that monitors plans and triggers executions.
- **DEX Integration:** Uniswap V3 for on-chain swaps with slippage protection.
- **Interface:** Telegram Bot for managing plans effortlessly.

Smart Contract Logic

- **Features:**
  a. Create, Pause, Resume, and Cancel plans.
  b. Top-up budget or withdraw unused funds.
- **Execution:**
  a. Uses `exactInputSingle` on Uniswap V3.
  b. Off-chain `minAmountOut` calculation for slippage safety.

User Flow & Reliability

- **Workflow:**
  a. User configures plan in Telegram.
  b. Budget is deposited into the contract.
  c. Executor checks readiness and executes swaps on schedule.
- **Reliability:** Covered by comprehensive unit tests for both contract and backend.

Limitations & What can be done next

- **Current version:** Single user, Sepolia testnet, USDC/WETH pair.
- **Next Steps:**
  a. Support for any token pairs.
  b. Integration with Chainlink Automation.
  c. Web UI for easier management.